VOLUMEN 05 · MÁSTER DATA & IA

Python para datos I

El lenguaje: tipos, colecciones, control de flujo, funciones, errores, ficheros y módulos. Lo justo y necesario para trabajar con datos, explicado para alguien que ya programa en SQL.

PARA QUIÉN

Álvaro, con el SQL del Bloque 0 terminado. Viste JavaScript en el Volumen 1 y tienes el entorno de Python del Volumen 01 instalado.

CÓMO ESTÁ ESCRITO

Cada celda de código se ha ejecutado de verdad en un notebook con Python 3.12, y su salida aparece tal cual la verás en VS Code. Cuando algo funciona distinto que en JavaScript o en SQL, se dice.

INCLUYE

Seis tickets resueltos en Python puro (sin pandas, que llega en el Volumen 06), con la revisión de Elena, y una chuleta del lenguaje.

Preparado para Álvaro · Octubre de 2026 · Unos dos días de trabajo

Índice

PARTE 0 · ANTES DE EMPEZAR 3

1. Cómo trabajar con este volumen 4

PARTE 1 · EL LENGUAJE 6

2. Valores y tipos 7

3. Colecciones 12

4. Decisiones y bucles 16

5. Funciones 20

PARTE 2 · PROGRAMAS DE VERDAD 24

6. Errores y excepciones 25

7. Ficheros 28

8. Módulos 32

9. Python y DuckDB 35

10. Scripts 38

TICKETS · MAREJADA EN PYTHON 41

Tickets · Volumen 05 42

Soluciones · Volumen 05 44

APÉNDICES · PARA TENER A MANO 53

A. Chuleta de Python 54

B. Qué viene en el Volumen 06 56

PARTE 0

Antes de empezar

Por qué Python además de SQL, y cómo vas a trabajar con él: notebooks y scripts.

1. Cómo trabajar con este volumen

1. Cómo trabajar con este volumen

1.1 Por qué Python, si ya sabes SQL

SQL es el mejor lenguaje que existe para una cosa: preguntar a datos que ya están en tablas. Todo lo que has hecho en los cuatro volúmenes anteriores es eso. Pero el trabajo de un ingeniero de datos tiene mucho de lo que SQL no sabe hacer:

TAREA	CON SQL	CON PYTHON
Consultar, agregar, unir tablas	Lo mejor	Posible, pero peor
Leer un JSON anidado de una API web	Muy limitado	Natural
Repetir una tarea cada día, con condiciones y avisos	No	Natural
Mover ficheros, renombrar, comprimir, enviar	No	Natural
Machine learning, deep learning, IA generativa	No	Es **el** lenguaje
Notebooks de Fabric y Databricks	Una parte	La otra parte (PySpark, Bloque 3)

No compiten: se complementan. Un buen ingeniero de datos usa SQL para lo que SQL hace bien, y Python para todo lo demás, incluido lanzar el SQL. En este volumen aprendes el lenguaje; en el 06, las herramientas de datos de Python (pandas, ficheros, APIs).

1.2 Lo que ya sabes de JavaScript

En el Volumen 1 viste JavaScript. Python se parece mucho en las ideas (variables, funciones, listas, objetos) y se diferencia en la forma. Las tres diferencias que más vas a notar:

	JAVASCRIPT	PYTHON
Bloques de código	Llaves `{ }`	**Sangría** (cuatro espacios). No es estética: es la sintaxis
Final de línea	`;` (opcional)	Nada
Declarar variables	`let x = 5`	`x = 5`, sin palabra delante
Comentarios	`//`	`#`
Verdadero / falso / nada	`true`, `false`, `null`	`True`, `False`, `None` (con mayúscula)
Y, o, no	`&&`, `||`, `!`	`and`, `or`, `not`
Listas	`[1, 2, 3]` (arrays)	`[1, 2, 3]` (listas)
Objetos	`{nombre: "Bar"}`	`{"nombre": "Bar"}` (diccionarios: las claves con comillas)

1.3 Dónde vas a escribir: el notebook

Trabajarás sobre todo en **notebooks** dentro de VS Code, como en el capítulo 10 del Volumen 01: celdas de código que se ejecutan de una en una, con el resultado debajo. Para este volumen, crea uno nuevo:

1. Abre el proyecto: `cd ~/master-data-ia` y `code .`
2. `Cmd` + `Shift` + `P` → `Create: New Jupyter Notebook`.
3. Guárdalo como `notebooks/05_python.ipynb`.

4. Arriba a la derecha, **Select Kernel** → el Python del `.venv` del proyecto.

Cada recuadro verde de este volumen es una celda. Escríbela (no la copies), ejecútala con `Shift` + `Intro`, y compara con el recuadro oscuro de debajo, que es su salida real.

PYTHON · EN TU NOTEBOOK

```
print("Hola, Marejada")
```

RESULTADO

```
Hola, Marejada
```

1.4 Cómo responde un notebook

Una celda puede mostrar cosas de dos formas, y conviene distinguirlas desde el principio:

PYTHON · EN TU NOTEBOOK

```
precio = 41.65
cantidad = 6
print("Importe calculado")
precio * cantidad
```

RESULTADO

```
Importe calculado
249.89999999999998
```

`print(...)` escribe lo que le pidas, en el momento en que se ejecuta. Además, si la **última línea** de la celda es un valor (aquí, `precio * cantidad`), *el notebook lo muestra automáticamente debajo. Solo la última: si pusieras* `precio` `cantidad` en medio de la celda, se calcularía y no se vería. Es una comodidad del notebook; en un script (capítulo 10) solo se ve lo que imprimes.

Y fíjate en el resultado: `249.89999999999998`, no `249.9`. No es un error de Python. Es el mismo problema de los decimales que viste en JavaScript en el Volumen 1 y en el aviso de `DOUBLE` del Volumen 02 (7.6). Lo resolverás en el 2.3.

EL ORDEN DE LAS CELDAS

Un notebook recuerda todo lo que has ejecutado, **en el orden en que lo ejecutaste**, no en el orden en que aparece en pantalla. Si cambias una celda de arriba y no la vuelves a ejecutar, las de abajo siguen usando el valor antiguo. Cuando algo no cuadre, **Restart** (reiniciar el kernel) en la barra del notebook y **Run All** para ejecutarlo todo de arriba abajo. El número entre corchetes a la izquierda de cada celda (`[3]`) dice en qué orden se ejecutó.

PARTE 1

El lenguaje

Valores, colecciones, decisiones, bucles y funciones: las piezas con las que se construye cualquier programa.

2. Valores y tipos

3. Colecciones

4. Decisiones y bucles

5. Funciones

2. Valores y tipos

2.1 Los tipos básicos

En SQL, cada columna tiene un tipo (`BIGINT`, `VARCHAR`, `DATE`) y DuckDB te lo muestra debajo del nombre. En Python, cada **valor** tiene un tipo, y la función `type()` te lo dice:

PYTHON · EN TU NOTEBOOK

```
print(type(1212))
print(type(41.65))
print(type("Bar La Ola"))
print(type(True))
print(type(None))
```

RESULTADO

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
```

TIPO DE PYTHON	QUÉ GUARDA	EQUIVALENTE EN SQL
`int`	Enteros, de cualquier tamaño	`INTEGER`, `BIGINT`
`float`	Números con decimales (aproximados)	`DOUBLE`
`str`	Texto	`VARCHAR`
`bool`	`True` o `False`	`BOOLEAN`
`None`	La ausencia de valor	`NULL`

Una variable no tiene tipo: es solo un nombre que apunta a un valor. La misma variable puede apuntar a un número y después a un texto. Es cómodo y peligroso a la vez, y por eso más adelante verás cómo anotar los tipos (5.6).

2.2 Números y operaciones

PYTHON · EN TU NOTEBOOK

```
print(7 / 2)     # división: siempre da float
print(7 // 2)    # división entera
print(7 % 2)     # resto
print(2 ** 10)   # potencia
print(round(2.675, 2))
```

RESULTADO

```
3.5
3
1
1024
2.67
```

Las mismas operaciones que en DuckDB (Volumen 02, 6.2), con una diferencia: la potencia es `**`. Y fíjate en el último: `round(2.675, 2)` da `2.67`, no `2.68`. Es el mismo problema de los decimales aproximados: `2.675` no se puede guardar exactamente en binario y en realidad es `2.67499999...`. Con dinero, esto importa.

2.3 El dinero: Decimal

Para importes, Python tiene un tipo exacto, como el `DECIMAL` de silver:

PYTHON · EN TU NOTEBOOK

```
from decimal import Decimal

print(0.1 + 0.2)
print(Decimal("0.1") + Decimal("0.2"))
print(Decimal("41.65") * 6)
```

RESULTADO

```
0.30000000000000004
0.3
249.90
```

DESMONTANDO LA CONSULTA

`from decimal import Decimal`

`from decimal`	El módulo `decimal`, que viene con Python. Un módulo es un fichero de código con herramientas; los verás en el capítulo 8.
`import Decimal`	Trae el tipo `Decimal` para poder usarlo por su nombre.
`Decimal("0.1")`	Un decimal exacto. Se crea **a partir de un texto**: `Decimal(0.1)`, sin comillas, heredaría el error del `float` antes de convertirse.

En la práctica, los datos de Marejada llegarán casi siempre ya calculados desde DuckDB o pandas, y redondear al final es suficiente. Pero cuando escribas código que sume dinero a mano (un script de facturación, una conciliación), usa `Decimal`.

2.4 Texto

Los textos (*strings*) van entre comillas simples o dobles; da igual cuáles, mientras abras y cierres con las mismas. Y tienen muchos métodos, que se llaman con un punto:

PYTHON · EN TU NOTEBOOK

```
provincia = "  CANTABRIA "
print(provincia.strip())
print(provincia.strip().title())
print(provincia.lower())
print("CER-0007".replace("-", ""))
print("CER-0007".split("-"))
print(len("CER-0007"))
```

RESULTADO

```
CANTABRIA
Cantabria
  cantabria
CER0007
['CER', '0007']
8
```

Los mismos que usaste en SQL con otro nombre: `trim` es `strip`, `upper` y `lower` se llaman igual, `split_part` es `split` (que devuelve una lista con todos los trozos). Como en SQL, se encadenan: `provincia.strip().title()` primero quita los espacios y después pone la primera letra en mayúscula.

IDEA CLAVE

Los textos en Python no se modifican: `provincia.strip()` no cambia `provincia`, sino que **devuelve un texto nuevo**. Si quieres quedarte con él, guárdalo: `provincia = provincia.strip()`. Es el error más común de la primera semana.

2.5 f-strings: texto con valores dentro

Para construir un texto con valores, la forma moderna es una **f-string**: una `f` delante de las comillas y los valores entre llaves:

PYTHON · EN TU NOTEBOOK

```
cliente = "Bar La Ola"
pedidos = 47
ventas = 18234.567

print(f"{cliente} ha hecho {pedidos} pedidos")
print(f"Ventas: {ventas:.2f} €")
print(f"Ventas: {ventas:,.2f} €")
print(f"Media por pedido: {ventas / pedidos:.1f} €")
```

RESULTADO

```
Bar La Ola ha hecho 47 pedidos
Ventas: 18234.57 €
Ventas: 18,234.57 €
Media por pedido: 388.0 €
```

DESMONTANDO EL COMANDO

`f"Ventas: {ventas:,.2f} €"`

`f"..."`	Una f-string: lo que va entre llaves se calcula y se mete en el texto.
`{ventas}`	El valor de la variable.
`:.2f`	Un **formato**: con dos decimales (`f` de *fixed*). Solo cambia cómo se muestra, no el valor.
`:,`	Con separador de miles. Python usa el estilo inglés (coma para los miles y punto para los decimales); para el formato español, en el Volumen 06 verás cómo hacerlo.
`{ventas / pedidos:.1f}`	Dentro de las llaves puede ir cualquier expresión, no solo una variable.

2.6 Convertir entre tipos

Exactamente el `CAST` de SQL, con el nombre del tipo como función:

PYTHON · EN TU NOTEBOOK

```
print(int("42") + 1)
print(float("41,65".replace(",", ".")))
print(str(1212) + " clientes")
print(bool(""), bool("hola"), bool(0), bool(42))
```

RESULTADO

```
43
41.65
1212 clientes
False True False True
```

La segunda línea es la coma decimal de los CSV de Marejada: Python, como DuckDB sin `decimal_separator`, solo entiende el punto. Y si la conversión es imposible, error, igual que el `CAST` de SQL:

PYTHON · EN TU NOTEBOOK

```
int("cuarenta")
```

ERROR

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 int("cuarenta")

ValueError: invalid literal for int() with base 10: 'cuarenta'
```

Así se lee un error de Python, y conviene aprender desde ya: **de abajo arriba**. La última línea dice qué ha pasado (`ValueError`: un valor que no sirve, y por qué). Encima, dónde: la celda, la línea y una flecha `---->` que la señala. El número de `Cell In[...]` es el contador de ejecuciones de tu notebook, así que en el tuyo será otro. En el capítulo 6 verás cómo atrapar estos errores en lugar de que paren el programa.

2.7 Comparaciones y None

PYTHON · EN TU NOTEBOOK

```
precio = 41.65
print(precio > 40)
print(precio == 41.65)
print(precio != 41.65)
print(10 < precio <= 50)

email = None
print(email is None)
```

RESULTADO

```
True
True
False
True
True
```

Dos detalles que te diferencian de SQL. La igualdad se escribe `==` (con un solo `=` asignas). Y las comparaciones se encadenan: `10 < precio <= 50` es "entre 10 y 50". Para `None`, como para `NULL`, se usa `is None` y no `== None` (funciona, pero es la costumbre y es más seguro).

> **NOTA**
>
> A diferencia de SQL, `None == None` en Python es `True`. Python no tiene la lógica de tres valores del `NULL` (el "no se sabe" del Volumen 02, 4.6). Cuando pases datos de DuckDB a Python, los `NULL` llegan como `None`, y se comportan como un valor más.

3. Colecciones

3.1 Listas

Una **lista** es una secuencia ordenada de valores, entre corchetes. Es el tipo que más vas a usar:

PYTHON · EN TU NOTEBOOK

```
almacenes = ["Guarnizo", "Llanera"]
ventas_mes = [1187705, 1066618, 1351890, 1500290]

print(len(ventas_mes))
print(ventas_mes[0])
print(ventas_mes[-1])
print(ventas_mes[1:3])
print(sum(ventas_mes), max(ventas_mes), min(ventas_mes))
```

RESULTADO

```
4
1187705
1500290
[1066618, 1351890]
5106503 1500290 1066618
```

DESMONTANDO EL COMANDO

`ventas_mes[0]` · `ventas_mes[-1]` · `ventas_mes[1:3]`

`[0]`	El primer elemento. **Las posiciones empiezan en 0**, como en JavaScript y al revés que en las listas de DuckDB (`[...][1]` era el primero, Volumen 04, 6.2).
`[-1]`	El último. Los números negativos cuentan desde el final: `[-2]` es el penúltimo.
`[1:3]`	Un **trozo** (*slice*): desde la posición 1 hasta la 3, **sin incluir la 3**. Devuelve una lista nueva con dos elementos.
`sum, max, min, len`	Funciones que reciben la lista entera. Los agregados de SQL, para una lista.

Las listas sí se pueden modificar:

PYTHON · EN TU NOTEBOOK

```
almacenes.append("Bilbao")
print(almacenes)
almacenes.remove("Bilbao")
print(almacenes)
print("Llanera" in almacenes)
```

RESULTADO

```
['Guarnizo', 'Llanera', 'Bilbao']
['Guarnizo', 'Llanera']
True
```

`append` añade al final, `remove` quita un valor, e `in` pregunta si un valor está: es el `IN` de SQL.

3.2 La trampa de las listas compartidas

Esto sorprende a todo el mundo la primera vez:

PYTHON · EN TU NOTEBOOK

```
provincias = ["Cantabria", "Asturias"]
copia = provincias
copia.append("Bizkaia")
print(provincias)
```

RESULTADO

```
['Cantabria', 'Asturias', 'Bizkaia']
```

Has añadido Bizkaia a `copia`, y aparece también en `provincias`. Porque `copia = provincias` **no copia la lista**: crea un segundo nombre para la misma lista. Las dos variables apuntan al mismo sitio. Para una copia de verdad:

PYTHON · EN TU NOTEBOOK

```
provincias = ["Cantabria", "Asturias"]
copia = provincias.copy()
copia.append("Bizkaia")
print(provincias)
print(copia)
```

RESULTADO

```
['Cantabria', 'Asturias']
['Cantabria', 'Asturias', 'Bizkaia']
```

Con números y textos no pasa, porque no se pueden modificar. Con listas y diccionarios (3.4), sí, y causa errores muy difíciles de encontrar.

3.3 Tuplas

Una **tupla** es como una lista que no se puede modificar, entre paréntesis. La vas a ver mucho por una razón: es lo que devuelve DuckDB cuando pides filas desde Python (capítulo 9). Cada fila es una tupla:

PYTHON · EN TU NOTEBOOK

```
fila = (10001, "Bar La Ola", "Cantabria")
print(fila[1])
cliente_id, nombre, provincia = fila
print(nombre, "-", provincia)
```

RESULTADO

```
Bar La Ola
Bar La Ola - Cantabria
```

La segunda forma se llama **desempaquetar**: tres variables a la izquierda reciben, en orden, los tres valores de la tupla. Lo usarás constantemente para recorrer resultados de una consulta.

3.4 Diccionarios

Un **diccionario** guarda parejas de **clave** y **valor**, entre llaves. Es el objeto de JavaScript y, sobre todo, la forma de los JSON:

PYTHON · EN TU NOTEBOOK

```
cliente = {
    "cliente_id": 10001,
    "nombre": "Bar La Ola",
    "provincia": "Cantabria",
    "activo": True,
}
print(cliente["nombre"])
cliente["telefono"] = "979940079"
print(cliente.keys())
print(len(cliente))
```

RESULTADO

```
Bar La Ola
dict_keys(['cliente_id', 'nombre', 'provincia', 'activo', 'telefono'])
5
```

Se accede por la clave entre corchetes, y asignar a una clave nueva la añade. Si pides una clave que no existe, error:

PYTHON · EN TU NOTEBOOK

```
cliente["email"]
```

ERROR

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[17], line 1
----> 1 cliente["email"]

KeyError: 'email'
```

`KeyError`: la clave no existe. Cuando no sabes si una clave está (en un JSON que viene de fuera, casi nunca lo sabes), usa `.get()`, que devuelve `None` (o el valor que le digas) en lugar de fallar:

PYTHON · EN TU NOTEBOOK

```
print(cliente.get("email"))
print(cliente.get("email", "(sin email)"))
```

RESULTADO

```
None
(sin email)
```

Es el `coalesce` de SQL aplicado a un diccionario.

3.5 Conjuntos

Un **conjunto** (*set*) es una colección sin orden y **sin repetidos**. Es el `DISTINCT` de Python:

PYTHON · EN TU NOTEBOOK

```
canales = ["Comercial", "Teléfono", "Comercial", "Web B2B", "Teléfono"]
print(set(canales))
print(len(set(canales)))

web_2025 = {10001, 10002, 10005}
web_2026 = {10002, 10005, 10007}
print(web_2026 - web_2025)   # están en 2026 y no en 2025
print(web_2025 & web_2026)   # están en los dos
```

RESULTADO

```
{'Web B2B', 'Teléfono', 'Comercial'}
3
{10007}
{10002, 10005}
```

El orden en que aparecen los elementos de un conjunto no está garantizado: no te fíes de él. Las operaciones de conjuntos son las del anti-join (`-`) y el `INNER JOIN` (`&`) del Volumen 02, para listas pequeñas que ya tienes en Python.

3.6 Cuál usar

COLECCIÓN	SINTAXIS	ORDENADA	MODIFICABLE	REPETIDOS	ÚSALA PARA
Lista	`[1, 2, 3]`	Sí	Sí	Sí	Casi todo: una serie de valores
Tupla	`(1, 2, 3)`	Sí	No	Sí	Un registro de tamaño fijo: una fila
Diccionario	`{"a": 1}`	Sí (orden de inserción)	Sí	Claves únicas	Buscar por nombre: un registro con campos, un JSON
Conjunto	`{1, 2, 3}`	No	Sí	No	Valores distintos y operaciones de conjuntos

4. Decisiones y bucles

4.1 if, elif, else

El `CASE` de SQL, como instrucción:

PYTHON · EN TU NOTEBOOK

```
gastos_envio = 18.0

if gastos_envio == 0:
    tipo = "Portes pagados"
elif gastos_envio == 12:
    tipo = "Portes Cantabria"
else:
    tipo = "Portes fuera de Cantabria"

print(tipo)
```

RESULTADO

```
Portes fuera de Cantabria
```

DESMONTANDO EL COMANDO

`if condición: · elif condición: · else:`

`if gastos_envio == 0:`	La condición, y **dos puntos** al final. Los dos puntos anuncian un bloque.
`(sangría de 4 espacios)`	Lo que se ejecuta si se cumple. Todo lo que está sangrado forma parte del bloque; el bloque acaba cuando la sangría vuelve atrás. Es lo que en JavaScript hacían las llaves.
`elif`	"Si no, si...". El `else if` de JavaScript. Puede haber tantos como quieras; se comprueban en orden y gana el primero que se cumpla, como en el `CASE`.
`else:`	Si no se cumple ninguna.

Si te equivocas con la sangría, Python no lo deja pasar:

PYTHON · EN TU NOTEBOOK

```
if gastos_envio > 0:
print("Paga portes")
```

ERROR

```
  Cell In[21], line 2
    print("Paga portes")
    ^
IndentationError: expected an indented block after 'if' statement on line 1
```

`IndentationError`: Python esperaba un bloque sangrado después de los dos puntos. VS Code sangra solo cuando pulsas `Intro` después de `:`, así que este error suele aparecer al copiar y pegar.

4.2 Lo que cuenta como verdadero

En un `if`, Python acepta cualquier valor, no solo `True` y `False`. Los "vacíos" cuentan como falsos: `0`, `""`, `[]`, `{}` y `None`. Todo lo demás, como verdadero. Es cómodo:

PYTHON · EN TU NOTEBOOK

```
email = ""
if email:
    print("Enviar a", email)
else:
    print("Cliente sin email")
```

RESULTADO

```
Cliente sin email
```

Pero tiene una trampa con los números: `if descuento:` sería falso con un descuento de `0`, que es un valor válido, no una ausencia. Cuando lo que quieres saber es si hay valor, pregunta exactamente eso: `if descuento is not None:`.

4.3 for: recorrer una colección

`for` recorre los elementos de cualquier colección, uno a uno:

PYTHON · EN TU NOTEBOOK

```
ventas_mes = {"2026-07": 2142228, "2026-08": 1975639, "2026-09": 1571643}

for mes, ventas in ventas_mes.items():
    print(f"{mes}: {ventas:>12,} €")
```

RESULTADO

```
2026-07:    2,142,228 €
2026-08:    1,975,639 €
2026-09:    1,571,643 €
```

`.items()` devuelve las parejas de clave y valor del diccionario, y el desempaquetado del 3.3 las separa en dos variables. El formato `>12,` alinea el número a la derecha en 12 posiciones: la columna sale recta.

Tres funciones que acompañan casi siempre a `for`:

PYTHON · EN TU NOTEBOOK

```
for i in range(3):
    print("Vuelta", i)

almacenes = ["Guarnizo", "Llanera"]
for posicion, almacen in enumerate(almacenes, start=1):
    print(posicion, almacen)

meses = ["jul", "ago", "sep"]
ventas = [2142228, 1975639, 1571643]
for mes, v in zip(meses, ventas):
    print(mes, v)
```

RESULTADO

```
Vuelta 0
Vuelta 1
Vuelta 2
1 Guarnizo
2 Llanera
jul 2142228
ago 1975639
sep 1571643
```

FUNCIÓN	QUÉ HACE	EQUIVALENTE QUE CONOCES
`range(3)`	Los números 0, 1, 2 (sin incluir el 3)	`generate_series` sin el final
`enumerate(lista, start=1)`	Cada elemento con su posición	`row_number()`
`zip(a, b)`	Junta dos listas elemento a elemento	Un `JOIN` por posición

4.4 Acumular dentro de un bucle

El patrón más típico: recorrer datos y acumular un resultado. Contar pedidos por canal, a mano:

PYTHON · EN TU NOTEBOOK

```
pedidos = [
    ("2026025160", "Web B2B"), ("2026025161", "Comercial"), ("2026025162", "Teléfono"),
    ("2026025163", "Comercial"), ("2026025164", "Web B2B"), ("2026025165", "Comercial"),
]

por_canal = {}
for pedido_id, canal in pedidos:
    por_canal[canal] = por_canal.get(canal, 0) + 1

print(por_canal)
```

RESULTADO

```
{'Web B2B': 2, 'Comercial': 3, 'Teléfono': 1}
```

Es un `GROUP BY canal` con `count(*)`, escrito paso a paso. `por_canal.get(canal, 0)` devuelve lo que llevas contado para ese canal, o 0 si es la primera vez que aparece. Ver cómo se hace a mano ayuda a entender lo que hace SQL por dentro; en la práctica, para esto existe `Counter` (8.3) y, para datos de verdad, DuckDB o pandas.

4.5 while, break y continue

`while` repite **mientras** se cumpla una condición. Se usa menos que `for` en datos, sobre todo para reintentar algo o paginar una API (Volumen 06):

PYTHON · EN TU NOTEBOOK

```
stock = 30
semanas = 0
while stock > 0:
    stock -= 8          # se venden 8 por semana
    semanas += 1
print(f"Se acaba en {semanas} semanas")
```

RESULTADO

```
Se acaba en 4 semanas
```

`stock -= 8` es la forma corta de `stock = stock - 8`. `break` sale del bucle en el acto y `continue` salta a la siguiente vuelta; los verás en los tickets.

EL BUCLE QUE NO TERMINA

Si la condición de un `while` no deja nunca de cumplirse (porque te olvidaste del `stock -= 8`), el bucle no termina y la celda se queda ejecutándose para siempre: lo verás porque el indicador de la celda no para de girar. Se detiene con el botón de **Interrupt** (el cuadrado) de la barra del notebook.

4.6 Comprensiones: un bucle en una línea

Construir una lista a partir de otra es tan común que Python tiene una sintaxis propia, la **comprensión de lista**:

PYTHON · EN TU NOTEBOOK

```
skus = [" cer-0012", "CER0007", "VIN-0025 ", "agu-0001"]

limpios = [s.strip().upper() for s in skus]
print(limpios)

solo_cervezas = [s for s in limpios if s.startswith("CER")]
print(solo_cervezas)
```

RESULTADO

```
['CER-0012', 'CER0007', 'VIN-0025', 'AGU-0001']
['CER-0012', 'CER0007']
```

DESMONTANDO EL COMANDO

`[s.strip().upper() for s in skus if condición]`

`for s in skus`	Recorre la lista original; en cada vuelta, el elemento se llama `s`.
`s.strip().upper()`	Lo que se mete en la lista nueva por cada elemento. Es el `SELECT` de la comprensión.
`if s.startswith("CER")`	Opcional: solo los que cumplan la condición. Es el `WHERE`.
`[ ... ]`	El resultado es una lista nueva. La original no cambia.

Se leen casi como una consulta: "dame `upper(trim(s))` de cada `s` en `skus` donde...". Con llaves en lugar de corchetes, se construye un diccionario: `{s: len(s) for s in limpios}`.

5. Funciones

5.1 Definir una función

Una **función** es un trozo de código con nombre que recibe datos y devuelve un resultado. La normalización del SKU del inventario, que en silver escribiste en SQL:

PYTHON · EN TU NOTEBOOK

```
def normalizar_sku(sku):
    """Devuelve el SKU en el formato del maestro: CER-0001."""
    limpio = sku.strip().upper().replace("-", "")
    return limpio[:3] + "-" + limpio[3:]


print(normalizar_sku(" cer-0012"))
print(normalizar_sku("CER0007"))
print(normalizar_sku("VIN-0025"))
```

RESULTADO

```
CER-0012
CER-0007
VIN-0025
```

DESMONTANDO EL COMANDO

`def normalizar_sku(sku): """..."""  ... return ...`

`def`	Define una función. Es el `function` de JavaScript.
`normalizar_sku`	Su nombre. En Python se escriben en minúsculas con guiones bajos (*snake_case*), igual que las columnas de tus tablas.
`(sku)`	El **parámetro**: el nombre con el que la función llama a lo que le pasen.
`"""..."""`	La **docstring**: una frase que explica qué hace. VS Code la muestra cuando pasas el ratón sobre el nombre de la función.
`limpio[:3] + "-" + limpio[3:]`	Las tres primeras letras, un guion y el resto. Los trozos del 3.1, aplicados a un texto: los textos también se recortan con `[ : ]`.
`return`	Lo que devuelve. Sin `return`, la función devuelve `None`.

Es la regla de `silver.inventario` en Python: `upper(replace(trim(sku), '-', ''))` y reconstruir con el guion. Escrita como función, se usa en cualquier sitio y se prueba por separado (MJD-502).

5.2 Parámetros con valor por defecto y por nombre

PYTHON · EN TU NOTEBOOK

```
def importe_neto(cantidad, precio, descuento_pct=0):
    return cantidad * precio * (1 - descuento_pct / 100)


print(importe_neto(6, 41.65))
print(importe_neto(6, 41.65, 10))
print(importe_neto(precio=41.65, cantidad=6, descuento_pct=10))
```

RESULTADO

```
249.89999999999998
224.91
224.91
```

`descuento_pct=0` hace el parámetro opcional: si no lo pasas, vale 0. Y al llamar se puede decir el nombre de cada parámetro, en cualquier orden. En funciones con muchos parámetros, nombrarlos hace el código mucho más legible: `importe_neto(6, 41.65, 10)` obliga a recordar qué era el 10.

5.3 Devolver varios valores

Una función puede devolver una tupla, y quien la llama la desempaqueta:

PYTHON · EN TU NOTEBOOK

```
def resumen(valores):
    return min(valores), max(valores), sum(valores) / len(valores)


minimo, maximo, media = resumen([1187705, 1066618, 1351890, 1500290])
print(f"Mínimo {minimo:,} · máximo {maximo:,} · media {media:,.0f}")
```

RESULTADO

```
Mínimo 1,066,618 · máximo 1,500,290 · media 1,276,626
```

5.4 Ámbito: lo que se ve dentro y fuera

Las variables que se crean dentro de una función solo existen dentro:

PYTHON · EN TU NOTEBOOK

```
def calcular():
    total_interno = 100
    return total_interno


calcular()
print(total_interno)
```

ERROR

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[31], line 7
      3     return total_interno
      4
      5
      6 calcular()
----> 7 print(total_interno)

NameError: name 'total_interno' is not defined
```

`NameError`: fuera de la función, `total_interno` no existe. Es una ventaja: una función no deja variables sueltas que puedan chocar con otras. Lo que quieras sacar de una función, sácalo con `return`.

5.5 La trampa del valor por defecto modificable

Una de las trampas más famosas de Python, y pregunta clásica de entrevista:

PYTHON · EN TU NOTEBOOK

```
def anadir_aviso(aviso, avisos=[]):
    avisos.append(aviso)
    return avisos


print(anadir_aviso("Stock bajo"))
print(anadir_aviso("Pedido urgente"))
```

RESULTADO

```
['Stock bajo']
['Stock bajo', 'Pedido urgente']
```

La segunda llamada debería devolver solo `['Pedido urgente']`, y arrastra el aviso de la primera. El valor por defecto `[]` se crea **una sola vez**, cuando se define la función, y todas las llamadas comparten esa misma lista (la trampa del 3.2, escondida). La forma correcta:

PYTHON · EN TU NOTEBOOK

```
def anadir_aviso(aviso, avisos=None):
    if avisos is None:
        avisos = []
    avisos.append(aviso)
    return avisos


print(anadir_aviso("Stock bajo"))
print(anadir_aviso("Pedido urgente"))
```

RESULTADO

```
['Stock bajo']
['Pedido urgente']
```

IDEA CLAVE

Nunca uses una lista o un diccionario como valor por defecto de un parámetro. Usa `None` y crea la lista dentro de la función.

5.6 Anotaciones de tipo

Python no obliga a declarar tipos, pero permite **anotarlos**:

PYTHON · EN TU NOTEBOOK

```
def margen_pct(ventas: float, coste: float) -> float:
    """Margen bruto en porcentaje."""
    return 100 * (1 - coste / ventas)


margen_pct(3947915, 2576130)
```

RESULTADO

```
34.74707535496585
```

`ventas: float` dice que se espera un número; `-> float`, que devuelve uno. Python **no lo comprueba** al ejecutar: es documentación que VS Code usa para avisarte, en el editor, si pasas un texto donde va un número. En el código profesional se usan casi siempre, y es una buena costumbre desde el principio.

5.7 Funciones como valores: ordenar por un criterio

Las funciones se pueden pasar a otras funciones. El uso más frecuente es decirle a `sorted` por qué ordenar:

PYTHON · EN TU NOTEBOOK

```
clientes = [("Bar La Ola", 18234), ("Hotel Mareas", 130249), ("Bar La Ría", 5422)]

print(sorted(clientes))
print(sorted(clientes, key=lambda c: c[1], reverse=True))
```

RESULTADO

```
[('Bar La Ola', 18234), ('Bar La Ría', 5422), ('Hotel Mareas', 130249)]
[('Hotel Mareas', 130249), ('Bar La Ola', 18234), ('Bar La Ría', 5422)]
```

Sin `key`, `sorted` ordena las tuplas por su primer elemento (el nombre). Con `key=lambda c: c[1]`, por el segundo (las ventas). `lambda c: c[1]` es una función pequeña sin nombre, escrita en línea: "dado `c`, devuelve `c[1]`". Es la flecha `c => c[1]` de JavaScript. `reverse=True` es el `DESC`.

PARTE 2

Programas de verdad

Errores, ficheros, módulos, DuckDB desde Python y scripts que se ejecutan solos.

6. Errores y excepciones

7. Ficheros

8. Módulos

9. Python y DuckDB

10. Scripts

6. Errores y excepciones

6.1 Leer un traceback

Ya has visto varios errores: `ValueError`, `KeyError`, `NameError`, `IndentationError`. En Python se llaman **excepciones**, y cuando ocurren dentro de funciones que llaman a otras funciones, el mensaje (el **traceback**) muestra el camino completo:

PYTHON · EN TU NOTEBOOK

```
def precio_unitario(texto):
    return float(texto)


def importe(cantidad, texto_precio):
    return cantidad * precio_unitario(texto_precio)


importe(6, "41,65")
```

ERROR

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[36], line 9
      5 def importe(cantidad, texto_precio):
      6     return cantidad * precio_unitario(texto_precio)
      7
      8
----> 9 importe(6, "41,65")

Cell In[36], line 6, in importe(cantidad, texto_precio)
      5 def importe(cantidad, texto_precio):
----> 6     return cantidad * precio_unitario(texto_precio)

Cell In[36], line 2, in precio_unitario(texto)
      1 def precio_unitario(texto):
----> 2     return float(texto)

ValueError: could not convert string to float: '41,65'
```

De abajo arriba otra vez: el error es que `"41,65"` no se puede convertir en número (la coma decimal). Ocurrió en la línea 2, dentro de `precio_unitario`; que fue llamada desde la línea 6, dentro de `importe`; que fue llamada desde la línea 9. El traceback es el camino del error, y la última flecha es donde ocurrió de verdad. La primera que es **tuya** (no de una librería) suele ser donde hay que arreglarlo.

6.2 try y except

Cuando sabes que algo puede fallar y sabes qué hacer si falla, lo atrapas:

PYTHON · EN TU NOTEBOOK

```
precios_ficheros = ["41.65", "41,65", "", "N/D", "102.00"]

for texto in precios_ficheros:
    try:
        precio = float(texto.replace(",", "."))
        print(f"{texto!r:>9} -> {precio}")
    except ValueError:
        print(f"{texto!r:>9} -> no es un precio")
```

RESULTADO

```
  '41.65' -> 41.65
  '41,65' -> 41.65
       '' -> no es un precio
    'N/D' -> no es un precio
 '102.00' -> 102.0
```

DESMONTANDO EL COMANDO

`try: ... except ValueError: ...`

`try:`	Intenta lo que va dentro.
`except ValueError:`	Si dentro ocurre un `ValueError`, no se para el programa: se ejecuta este bloque en su lugar. Si ocurre otro tipo de error, no lo atrapa y el programa se para igual.
`{texto!r}`	En una f-string, `!r` muestra el valor como se escribiría en Python (con comillas). Así se ve que `''` es un texto vacío.

Es el `TRY_CAST` de SQL (Volumen 02, 6.2) hecho a mano, con la ventaja de que puedes decidir qué hacer con cada fallo: contarlo, apuntarlo, avisar.

NUNCA EXCEPT A SECAS

`except:` sin tipo, o `except Exception:`, atrapa **todos** los errores, incluidos los que no esperabas (una errata en el nombre de una variable, por ejemplo). El programa sigue como si nada y el error queda escondido. Atrapa siempre el tipo concreto que esperas, y deja que lo inesperado pare el programa: un error que se ve es mejor que un resultado silenciosamente mal.

6.3 Lanzar tus propios errores

Cuando una función recibe algo que no tiene sentido, lo correcto es pararse con un mensaje claro, no seguir y devolver basura:

PYTHON · EN TU NOTEBOOK

```python
def margen_pct(ventas, coste):
    if ventas <= 0:
        raise ValueError(f"Las ventas deben ser positivas y son {ventas}")
    return 100 * (1 - coste / ventas)


margen_pct(0, 150)
```

ERROR

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[38], line 7
      3         raise ValueError(f"Las ventas deben ser positivas y son {ventas}")
      4     return 100 * (1 - coste / ventas)
      5
      6
----> 7 margen_pct(0, 150)

Cell In[38], line 3, in margen_pct(ventas, coste)
      1 def margen_pct(ventas, coste):
      2     if ventas <= 0:
----> 3         raise ValueError(f"Las ventas deben ser positivas y son {ventas}")
      4     return 100 * (1 - coste / ventas)

ValueError: Las ventas deben ser positivas y son 0
```

`raise` lanza la excepción. Sin esa comprobación, `margen_pct(0, 150)` daría un `ZeroDivisionError`, que dice lo que ha pasado pero no por qué; con ella, el mensaje explica el problema en el idioma del negocio.

7. Ficheros

7.1 Rutas con pathlib

Los ficheros se localizan por su **ruta**. El notebook está en `notebooks/`, así que los datos están un nivel más arriba, en `../datos/`. Python tiene un módulo para trabajar con rutas sin pelearse con las barras, `pathlib`:

PYTHON · EN TU NOTEBOOK

```
from pathlib import Path

datos = Path("../datos")
print(datos.exists())

for fichero in sorted(datos.rglob("*.csv")):
    print(f"{fichero.stat().st_size:>12,}  {fichero}")
```

RESULTADO

```
True
      31,907  ../datos/almacen/inventario_2026-09-30.csv
     184,713  ../datos/erp/clientes.csv
  13,284,564  ../datos/erp/lineas_pedido.csv
   6,436,477  ../datos/erp/pedidos.csv
      28,095  ../datos/erp/productos.csv
       1,541  ../datos/rrhh/comerciales.csv
```

DESMONTANDO EL COMANDO

`Path("../datos").rglob("*.csv")`

`Path("../datos")`	Un objeto ruta. No abre nada: solo representa una ruta, exista o no.
`.exists()`	¿Existe?
`.rglob("*.csv")`	Busca, en la carpeta y en todas sus subcarpetas, los ficheros que encajen con el patrón. Es el `find datos -name "*.csv"` del Volumen 1.
`.stat().st_size`	El tamaño en bytes.
`datos / "erp" / "pedidos.csv"`	Las rutas se unen con `/`. Funciona igual en Mac y en Windows (donde la barra es la otra): por eso se usa `pathlib` y no se juntan textos a mano.

7.2 Abrir y leer un fichero de texto

PYTHON · EN TU NOTEBOOK

```
ruta = Path("../datos/LEEME.md")
with open(ruta, encoding="utf-8") as f:
    lineas = f.readlines()

print(len(lineas), "líneas")
print(lineas[0])
```

RESULTADO

```
19 líneas
# Datos de Marejada Distribución, S.L. · versión 1
```

DESMONTANDO LA CONSULTA

`with open(ruta, encoding="utf-8") as f:`

`open(ruta, ...)`	Abre el fichero para leer. Devuelve un objeto fichero.
`encoding="utf-8"`	En qué codificación está. Escríbelo **siempre**: si no, Python usa la del sistema, que no siempre es UTF-8.
`with ... as f:`	Abre el fichero, lo llama `f` dentro del bloque, y lo **cierra al salir** del bloque, aunque haya un error. Es la forma correcta de abrir ficheros.
`f.readlines()`	Todas las líneas, como una lista de textos. Cada una termina con `\n`, el salto de línea; para quitarlo, `.rstrip()` (7.3).

7.3 La codificación, otra vez

El fichero de RRHH del Volumen 01 (8.5), en Latin-1. Si lo abres como UTF-8:

PYTHON · EN TU NOTEBOOK

```
with open("../datos/rrhh/comerciales.csv", encoding="utf-8") as f:
    texto = f.read()
```

ERROR

```
---------------------------------------------------------------------------
UnicodeDecodeError                        Traceback (most recent call last)
Cell In[41], line 2
      1 with open("../datos/rrhh/comerciales.csv", encoding="utf-8") as f:
----> 2     texto = f.read()

File <frozen codecs>:322, in BufferedIncrementalDecoder.decode(self, input, final)
    319 'Could not get source, probably due dynamically evaluated source code.'

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xcd in position 80: invalid continuation byte
```

`UnicodeDecodeError`: el byte `0xcd` no es UTF-8 válido. Es la `Í` de Íñigo en Latin-1. Las líneas del medio (`<frozen codecs>`) son el código interno de Python donde se detectó el problema: puedes ignorarlas, lo tuyo es la línea de la flecha y la última. El mismo problema que viste con DuckDB, con la misma solución: decir la codificación correcta.

PYTHON · EN TU NOTEBOOK

```
with open("../datos/rrhh/comerciales.csv", encoding="latin-1") as f:
    for linea in f.readlines()[:3]:
        print(linea.rstrip())
```

RESULTADO

```
comercial_id,nombre,apellidos,email,zona,puesto,jefe_id,fecha_alta,fecha_baja
1,Íñigo,Sainz Ruiz,inigo.sainz@marejada-distribucion.es,,Director comercial,,2015-03-02,
2,Raquel,Fernández Cobo,raquel.fernandez@marejada-distribucion.es,Cantabria Centro,Jefa de ventas
Cantabria,1,2016-09-12,
```

7.4 CSV con el módulo csv

Un CSV parece fácil de leer partiendo cada línea por las comas, pero no lo es: un campo puede llevar comas dentro si va entre comillas, y las líneas vacías o los saltos de línea dentro de un campo lo complican. Python trae un módulo que lo hace bien, `csv`. Su `DictReader` devuelve cada fila como un diccionario, con los nombres de la cabecera como claves:

PYTHON · EN TU NOTEBOOK

```
import csv

with open("../datos/rrhh/comerciales.csv", encoding="latin-1", newline="") as f:
    comerciales = list(csv.DictReader(f))

print(len(comerciales))
print(comerciales[1])
```

RESULTADO

```
14
{'comercial_id': '2', 'nombre': 'Raquel', 'apellidos': 'Fernández Cobo', 'email':
'raquel.fernandez@marejada-distribucion.es', 'zona': 'Cantabria Centro', 'puesto': 'Jefa de ventas
Cantabria', 'jefe_id': '1', 'fecha_alta': '2016-09-12', 'fecha_baja': ''}
```

Fíjate en dos cosas. `newline=""` es lo que pide la documentación del módulo `csv` al abrir el fichero: así gestiona él los saltos de línea. Y **todos los valores son texto**: `'1'`, `'2016-09-12'`, `''`. El módulo `csv` no adivina tipos (al revés que DuckDB): los números y las fechas los conviertes tú. Es uno de los motivos por los que para datos de verdad se usa pandas o DuckDB.

Para los ficheros del ERP, con punto y coma, se le dice el separador:

PYTHON · EN TU NOTEBOOK

```
with open("../datos/erp/productos.csv", encoding="utf-8", newline="") as f:
    lector = csv.DictReader(f, delimiter=";")
    primera = next(lector)

print(primera["sku"], primera["nombre"], primera["precio_lista"])
```

RESULTADO

```
CER-0001 Faro Norte Lager 102,00
```

`next(lector)` lee solo la primera fila, sin cargar el fichero entero. El precio, `'102,00'`: texto con coma decimal.

7.5 Escribir un fichero

Igual que leer, con `"w"` (*write*):

PYTHON · EN TU NOTEBOOK

```
Path("../entregables").mkdir(exist_ok=True)

with open("../entregables/prueba_python.txt", "w", encoding="utf-8") as f:
    f.write("Informe generado desde Python\n")
    f.write(f"Comerciales en RRHH: {len(comerciales)}\n")

print(Path("../entregables/prueba_python.txt").read_text(encoding="utf-8"))
```

RESULTADO

```
Informe generado desde Python
Comerciales en RRHH: 14
```

`"w"` crea el fichero o **lo sobrescribe entero** si ya existe. Para añadir al final sin borrar, `"a"` (*append*). Y `mkdir(exist_ok=True)` crea la carpeta si no existe, sin error si ya está: el `mkdir -p` de la terminal.

BORRA EL FICHERO DE PRUEBA

`entregables/prueba_python.txt` es solo una prueba: bórralo (`rm entregables/prueba_python.txt`) para que no acabe en tu repositorio.

7.6 JSON

El formato de la tienda web. El módulo `json` convierte un JSON en diccionarios y listas de Python:

PYTHON · EN TU NOTEBOOK

```
import json

with open("../datos/web/pedidos_web.json", encoding="utf-8") as f:
    web = json.load(f)

print(type(web), list(web.keys()))
print(web["count"], len(web["orders"]))
print(web["orders"][0]["customer"]["name"])
print(web["orders"][0]["items"][0])
```

RESULTADO

```
<class 'dict'> ['source', 'exported_at', 'count', 'orders']
7956 7956
Bar La Castellana
{'sku': 'AGU-0014', 'qty': 1, 'unit_price': 9.13, 'discount': 5.0}
```

Lo que en DuckDB necesitaba `unnest` y la sintaxis de los `STRUCT` (Volumen 01, MJD-100), aquí es navegar con corchetes: `web["orders"][0]` es el primer pedido, `["customer"]["name"]` entra en el cliente y saca el nombre. Un JSON es exactamente un diccionario de listas de diccionarios, y Python lo lee tal cual. Lo usarás en el MJD-505.

8. Módulos

8.1 La biblioteca estándar

Un **módulo** es un fichero de código de Python que otro programa puede usar con `import`. Python trae cientos de serie: la **biblioteca estándar**. Ya has usado `decimal`, `pathlib`, `csv` y `json`. Tres formas de importar:

FORMA	QUÉ HACE	SE USA ASÍ
`import json`	Trae el módulo entero	`json.load(f)`
`from pathlib import Path`	Trae solo una cosa	`Path(...)`
`import datetime as dt`	Trae el módulo con otro nombre, más corto	`dt.date(2026, 9, 30)`

8.2 Fechas: datetime

PYTHON · EN TU NOTEBOOK

```
import datetime as dt

corte = dt.date(2026, 9, 30)
print(corte)
print(corte - dt.timedelta(days=90))
print(corte.isoweekday(), corte.strftime("%d/%m/%Y"))

fecha_pedido = dt.datetime.strptime("01/01/2024 08:03", "%d/%m/%Y %H:%M")
print(fecha_pedido, fecha_pedido.year)
print((corte - fecha_pedido.date()).days, "días desde el primer pedido")
```

RESULTADO

```
2026-09-30
2026-07-02
3 30/09/2026
2024-01-01 08:03:00 2024
1003 días desde el primer pedido
```

Todo lo que hiciste con fechas en SQL tiene su equivalente: `timedelta(days=90)` es el `INTERVAL 90 DAY`; `isoweekday()` es el `isodow`; `strftime` convierte una fecha en texto con las mismas letras `%` del Volumen 01, y `strptime` (de *parse*) hace lo contrario. Restar dos fechas da un `timedelta`, y `.days` saca los días.

8.3 Contar: collections.Counter

El `GROUP BY ... count(*)` que escribiste a mano en el 4.4, ya hecho:

PYTHON · EN TU NOTEBOOK

```
from collections import Counter

estados_web = Counter(pedido["status"] for pedido in web["orders"])
print(estados_web)
print(estados_web.most_common(2))
print(estados_web["sync_error"])
```

RESULTADO

```
Counter({'completed': 7543, 'cancelled': 297, 'shipped': 49, 'processing': 36, 'sync_error': 31})
[('completed', 7543), ('cancelled', 297)]
31
```

Ahí están los 31 pedidos web con `sync_error` que nunca llegaron al ERP: el mismo número que dice la documentación del máster. Fíjate en lo que va dentro de `Counter(...)`: una comprensión sin corchetes. Se llama **expresión generadora** y hace lo mismo que la comprensión de lista, pero sin construir la lista en memoria: va pasando los valores uno a uno.

8.4 Tu propio módulo

Las funciones que vas a usar en varios notebooks no se copian de uno a otro: se escriben una vez en un fichero `.py` y se importan. Crea `notebooks/marejada_utils.py` con VS Code:

FICHERO · notebooks/marejada_utils.py

```
"""Utilidades de Marejada: funciones que se usan en varios notebooks."""
from decimal import Decimal

FECHA_CORTE = "2026-09-30"


def normalizar_sku(sku: str) -> str:
    """Devuelve el SKU en el formato del maestro: CER-0001."""
    limpio = sku.strip().upper().replace("-", "")
    return limpio[:3] + "-" + limpio[3:]


def a_decimal(texto: str) -> Decimal:
    """Convierte un importe del ERP ('41,65') en Decimal."""
    return Decimal(texto.strip().replace(",", "."))
```

Y en el notebook:

PYTHON · EN TU NOTEBOOK

```
import marejada_utils as mu

print(mu.FECHA_CORTE)
print(mu.normalizar_sku("cer0012"))
print(mu.a_decimal("102,00") * 3)
```

RESULTADO

```
2026-09-30
CER-0012
306.00
```

`import marejada_utils` busca un fichero `marejada_utils.py` en la carpeta del notebook, y por eso funciona. `FECHA_CORTE` en mayúsculas es la convención de Python para una **constante**: un valor que no debe cambiar. La fecha de corte de Marejada escrita una sola vez, como la regla del Volumen 02.

SI CAMBIAS EL MÓDULO

Python solo lee un módulo la primera vez que lo importas en una sesión. Si cambias `marejada_utils.py` y vuelves a ejecutar el `import`, el notebook **sigue usando la versión antigua**. Reinicia el kernel (**Restart**) después de cambiar un módulo, o el notebook te dará resultados de un código que ya no existe.

8.5 Paquetes de fuera: uv add

Lo que no está en la biblioteca estándar se instala. Ya lo hiciste en el Volumen 01 con `uv add duckdb pandas ...`. Para el Volumen 06 vas a necesitar uno más, `requests`, para llamar a APIs web. Desde la terminal, en la raíz del proyecto:

TÚ ESCRIBES

```
uv add requests
```

Queda apuntado en `pyproject.toml` y en `uv.lock`, y al hacer commit de esos dos ficheros, cualquiera que clone tu repositorio tendrá exactamente las mismas versiones con `uv sync`. Si el notebook no lo encuentra justo después de instalarlo, reinicia el kernel.

9. Python y DuckDB

9.1 Conectar y consultar

La conexión del Volumen 01, ahora entendiendo cada pieza:

PYTHON · EN TU NOTEBOOK

```
import duckdb

con = duckdb.connect("../db/marejada.duckdb", read_only=True)
con.sql("SELECT count(*) AS lineas, round(sum(importe_neto)) AS ventas FROM gold.fact_ventas")
```

RESULTADO

```
┌─────────┬──────────────┐
│ lineas  │    ventas    │
│ int64   │    double    │
├─────────┼──────────────┤
│ 366928  │ 42633273.0   │
└─────────┴──────────────┘
```

`con.sql(...)` devuelve un objeto **relación** de DuckDB, que el notebook pinta con las cajas de siempre. Para verlo es perfecto; para trabajar con los datos en Python, hay que sacarlos.

9.2 Sacar los datos: fetchall y fetchone

PYTHON · EN TU NOTEBOOK

```
filas = con.sql("""
    SELECT c.provincia, round(sum(f.importe_neto)) AS ventas
    FROM gold.fact_ventas AS f
    JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
    JOIN gold.dim_fecha   AS d ON d.fecha_key   = f.fecha_key
    WHERE d.anio = 2026
    GROUP BY c.provincia
    ORDER BY ventas DESC
""").fetchall()

print(type(filas), len(filas))
print(filas[0])

for provincia, ventas in filas:
    print(f"{provincia:<12} {ventas:>14,.0f} €")
```

RESULTADO

```
<class 'list'> 5
('Cantabria', 7985003.0)
Cantabria         7,985,003 €
Asturias          2,536,086 €
Bizkaia           1,635,110 €
Burgos            1,284,843 €
Palencia            742,344 €
```

DESMONTANDO EL COMANDO

`con.sql(""" ... """).fetchall()`

`""" ... """`	Un texto de varias líneas, entre comillas triples. Así una consulta larga se escribe igual que en un fichero `.sql`.
`.fetchall()`	Ejecuta la consulta y devuelve **una lista de tuplas**: una tupla por fila (3.3), con los valores en el orden de las columnas del `SELECT`.
`for provincia, ventas in filas:`	Recorre las filas desempaquetando cada tupla en sus dos columnas.
`{provincia:<12}`	Alinea el texto a la izquierda en 12 posiciones, para que la columna de importes salga recta.

Si la consulta devuelve una sola fila, `.fetchone()` devuelve esa tupla directamente:

PYTHON · EN TU NOTEBOOK

```
total, = con.sql("SELECT sum(importe_neto) FROM gold.fact_ventas").fetchone()
print(f"{total:,.2f}")
```

RESULTADO

```
42,633,273.34
```

La coma de `total, =` desempaqueta una tupla de un solo elemento. Sin ella, `total` sería una tupla con un número dentro, y el formato `:,.2f` daría error.

9.3 Parámetros: nunca metas valores con f-strings

Cuando una consulta depende de un valor que viene de fuera (una fecha que escribe alguien, un cliente elegido en una lista), la tentación es meterlo con una f-string. No lo hagas. Se pasan como **parámetros**:

PYTHON · EN TU NOTEBOOK

```
def ventas_de(provincia, anio):
    consulta = """
        SELECT round(sum(f.importe_neto)) AS ventas
        FROM gold.fact_ventas AS f
        JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
        JOIN gold.dim_fecha   AS d ON d.fecha_key   = f.fecha_key
        WHERE c.provincia = ? AND d.anio = ?
    """
    return con.execute(consulta, [provincia, anio]).fetchone()[0]


print(ventas_de("Bizkaia", 2025))
print(ventas_de("Bizkaia", 2026))
```

RESULTADO

```
691673.0
1635110.0
```

DESMONTANDO EL COMANDO

`con.execute(consulta, [provincia, anio])`

`?`	Un hueco para un valor. Cada `?` se rellena, en orden, con un elemento de la lista.
`[provincia, anio]`	Los valores. DuckDB los mete en la consulta **como datos**, nunca como código SQL.
`.execute(...)`	Como `.sql()`, pero admite parámetros. Después, `.fetchone()` o `.fetchall()`.

¿Por qué no `f"WHERE c.provincia = '{provincia}'"`? Porque si el valor viniera de un formulario y alguien escribiera `Bizkaia' OR '1'='1`, la consulta haría otra cosa: el texto se convertiría en código. Se llama **inyección de SQL** y es uno de los fallos de seguridad más antiguos y frecuentes. Con parámetros, el texto es siempre un dato. En un notebook tuyo no hay riesgo, pero la costumbre se coge ahora, porque en una aplicación de verdad sí lo hay.

9.4 Cerrar la conexión

PYTHON · EN TU NOTEBOOK

```
con.close()
```

Cerrar libera el fichero. Con `read_only=True` no bloquea a nadie, pero es una buena costumbre al terminar, y evita el error del candado del Volumen 01 (10.4) si alguna vez abres la base para escribir.

10. Scripts

10.1 Del notebook al script

Un notebook es ideal para explorar. Pero lo que se tiene que ejecutar solo (cada mañana, desde una tarea programada, en un servidor) es un **script**: un fichero `.py` que se ejecuta de principio a fin. Crea la carpeta `scripts/` en la raíz del proyecto y dentro `resumen_ventas.py`:

FICHERO · scripts/resumen_ventas.py

```
"""Resumen de ventas por año desde la capa gold.

Uso:  uv run python scripts/resumen_ventas.py
"""
import duckdb

RUTA_BD = "db/marejada.duckdb"


def ventas_por_anio(con):
    return con.sql("""
        SELECT d.anio, round(sum(f.importe_neto)) AS ventas
        FROM gold.fact_ventas AS f
        JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
        GROUP BY d.anio
        ORDER BY d.anio
    """).fetchall()


def main():
    con = duckdb.connect(RUTA_BD, read_only=True)
    for anio, ventas in ventas_por_anio(con):
        print(f"{anio}: {ventas:>14,.0f} €")
    con.close()


if __name__ == "__main__":
    main()
```

Se ejecuta desde la terminal, en la raíz del proyecto:

TÚ ESCRIBES

```
uv run python scripts/resumen_ventas.py
```

RESULTADO

```
2024:     12,830,511 €
2025:     15,619,376 €
2026:     14,183,385 €
```

DESMONTANDO EL COMANDO

`if __name__ == "__main__": main()`

`def main():`	Lo que hace el script, dentro de una función. Es la convención: todo el programa en `main`.
`__name__`	Una variable que Python rellena solo. Vale `"__main__"` cuando el fichero se **ejecuta** directamente, y el nombre del módulo cuando se **importa** desde otro sitio.
`if __name__ == "__main__":`	"Solo si me están ejecutando, no importando". Así puedes importar `ventas_por_anio` desde un notebook sin que se ejecute todo el script.
`RUTA_BD = "db/marejada.duckdb"`	Sin `../`: el script se ejecuta desde la raíz del proyecto, no desde `notebooks/`. Las rutas relativas dependen siempre de **desde dónde** se ejecuta, igual que en DuckDB (Volumen 01, 9.2).

10.2 Recibir datos desde la terminal

Un script útil suele necesitar algún dato de quien lo ejecuta: una fecha, una provincia. El módulo `argparse` lee lo que se escribe detrás del nombre del script y genera solo la ayuda:

FICHERO · scripts/ventas_provincia.py

```
"""Ventas de una provincia en un año, desde gold.

Uso:  uv run python scripts/ventas_provincia.py Cantabria --anio 2025
"""
import argparse

import duckdb


def main():
    parser = argparse.ArgumentParser(description="Ventas de una provincia en un año.")
    parser.add_argument("provincia", help="Provincia, como en dim_cliente (p. ej. Cantabria)")
    parser.add_argument("--anio", type=int, default=2026, help="Año (por defecto, 2026)")
    args = parser.parse_args()

    con = duckdb.connect("db/marejada.duckdb", read_only=True)
    ventas, = con.execute("""
        SELECT round(sum(f.importe_neto), 2)
        FROM gold.fact_ventas AS f
        JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
        JOIN gold.dim_fecha   AS d ON d.fecha_key   = f.fecha_key
        WHERE c.provincia = ? AND d.anio = ?
    """, [args.provincia, args.anio]).fetchone()
    con.close()

    if ventas is None:
        print(f"No hay ventas de {args.provincia} en {args.anio}.")
    else:
        print(f"{args.provincia} {args.anio}: {ventas:,.2f} €")


if __name__ == "__main__":
    main()
```

TÚ ESCRIBES

```
uv run python scripts/ventas_provincia.py Cantabria --anio 2025
uv run python scripts/ventas_provincia.py Galicia
uv run python scripts/ventas_provincia.py --help
```

RESULTADO

```
Cantabria 2025: 9,521,799.56 €
No hay ventas de Galicia en 2026.
usage: ventas_provincia.py [-h] [--anio ANIO] provincia

Ventas de una provincia en un año.

positional arguments:
  provincia    Provincia, como en dim_cliente (p. ej. Cantabria)

options:
  -h, --help   show this help message and exit
  --anio ANIO  Año (por defecto, 2026)
```

DESMONTANDO EL COMANDO

`parser.add_argument("provincia") · parser.add_argument("--anio", type=int, default=2026)`

`"provincia"`	Un argumento **obligatorio**, por posición: lo primero que se escribe detrás del script.
`"--anio"`	Un argumento **opcional**, con nombre. Si no se escribe, vale el `default`.
`type=int`	`argparse` lo convierte a número. Si escribes `--anio dos mil`, da un error claro antes de llegar a la consulta.
`--help`	Lo genera `argparse` solo, a partir de las descripciones (`help=`). Un script con `--help` es un script que otra persona puede usar.

Fíjate en el segundo caso: Galicia no existe en los datos, y en lugar de imprimir `None` el script lo dice. Pensar en qué pasa cuando la consulta no encuentra nada es la diferencia entre un script de prueba y uno que se puede dejar a otra persona.

TICKETS

Marejada en Python

Seis encargos resueltos solo con el lenguaje y la biblioteca estándar. Cada uno en su notebook o su script, y todo a Git.

Tickets · Volumen 05

Soluciones · Volumen 05

Tickets · Volumen 05

Para estos tickets, **sin pandas**: solo lo de este volumen. Los notebooks van a `notebooks/` con el nombre del ticket (`mjd-501_inventario.ipynb`) y los scripts a `scripts/`. La fecha de corte, como siempre, el 30 de septiembre de 2026.

TICKET MJD-501 De: Marcos Diego · Logística y almacén

Prioridad: media

Me han dicho que lo del MJD-102 (referencias por almacén) salió con SKU mal escritos. ¿Me lo repites bien? Y esta vez sin base de datos: el fichero de almacén tal cual llega, porque quiero poder hacerlo yo cada vez que lo exporte.

CRITERIOS DE ACEPTACIÓN

- Leer directamente `datos/almacen/inventario_2026-09-30.csv`.
- Referencias distintas (con el SKU normalizado) y unidades totales por almacén.
- Cuántas filas tenían el SKU mal escrito.

TICKET MJD-502 De: Elena Pérez · Responsable de datos

Prioridad: media

Tenemos la misma regla de normalización del SKU en dos sitios: en SQL (silver) y en Python (`marejada_utils.py`). Quiero estar segura de que hacen lo mismo. Pruébala con los casos raros y compárala con silver fila a fila.

TICKET MJD-503 De: Raquel Fernández · Jefa de ventas Cantabria

Prioridad: alta

Para la campaña de reactivación: un mensaje personalizado para cada cliente de Cantabria que lleve entre 90 y 180 días sin pedir, para que lo mande su comercial por WhatsApp. Con su nombre, cuánto lleva sin pedir y el nombre del comercial.

ENTREGABLE

`entregables/mjd-503_mensajes.txt`, un mensaje por cliente, separados por una línea en blanco.

TICKET MJD-504 De: Elena Pérez · Responsable de datos

Prioridad: baja

Para el manual de bienvenida de los nuevos: el organigrama comercial dibujado como un árbol de texto, a partir del fichero de RRHH. Sin base de datos.

TICKET MJD-505 De: Paula Herrero · Marketing y tienda online

Prioridad: alta

Me dice Elena que hay pedidos de la web que nunca llegaron al ERP. ¿Cuántos son, cuánto dinero suman, de cuántos clientes y desde cuándo pasa?

TICKET MJD-506 De: Íñigo Sainz · Director comercial

Prioridad: media

Quiero un informe cada lunes con las ventas de la semana anterior por canal, comparadas con la misma semana del año pasado. Que lo pueda lanzar cualquiera con una fecha.

ENTREGABLE

`scripts/informe_semana.py`, que reciba la fecha de un lunes y lo imprima.

Soluciones · Volumen 05

MJD-501 · Inventario por almacén, en Python

Leer con `csv.DictReader`, normalizar el SKU con la función de `marejada_utils`, y acumular por almacén. Para las referencias distintas, un **conjunto** por almacén (3.5); para las unidades, una suma:

PYTHON · EN TU NOTEBOOK

```
import csv
import marejada_utils as mu

referencias = {}      # almacén -> conjunto de SKU normalizados
unidades = {}         # almacén -> unidades totales
mal_escritos = 0

with open("../datos/almacen/inventario_2026-09-30.csv", encoding="utf-8", newline="") as f:
    for fila in csv.DictReader(f):
        almacen = fila["almacen"]
        sku = mu.normalizar_sku(fila["sku"])
        if sku != fila["sku"]:
            mal_escritos += 1
        referencias.setdefault(almacen, set()).add(sku)
        unidades[almacen] = unidades.get(almacen, 0) + int(fila["unidades"])

for almacen in sorted(referencias):
    print(f"{almacen:<10} {len(referencias[almacen]):>4} referencias  {unidades[almacen]:>7,}
unidades")
print(f"Filas con el SKU mal escrito: {mal_escritos}")
```

RESULTADO

```
Guarnizo    235 referencias   13,319 unidades
Llanera     146 referencias    8,130 unidades
Filas con el SKU mal escrito: 36
```

DESMONTANDO EL COMANDO

`referencias.setdefault(almacen, set()).add(sku)`

`setdefault(almacen, set())`	Devuelve el conjunto de ese almacén; si todavía no existe, lo crea vacío y lo devuelve. Evita el `if almacen not in referencias` de cada vuelta.
`.add(sku)`	Añade el SKU al conjunto. Si ya estaba, no pasa nada: un conjunto no tiene repetidos, y eso es justo "referencias distintas".
`int(fila["unidades"])`	El módulo `csv` lo da todo como texto (7.4). Sin `int`, el `+` **juntaría textos**: `"12" + "5"` es `"125"`.

Las mismas cifras que en el MJD-102: 235 referencias en Guarnizo y 146 en Llanera, con 13.319 y 8.130 unidades. Y 36 filas con el SKU mal escrito: son las 45 que no cruzaban con el maestro en el Volumen 02 (8.7) menos las 9 referencias `OBS-`, que están bien escritas pero no tienen ficha. Es decir: los SKU mal escritos no estaban creando referencias falsas. Ninguna referencia aparecía dos veces en el mismo almacén con dos grafías, y la advertencia de la revisión de Elena era prudencia, no un error.

REVISIÓN DE ELENA

Bien, y fíjate en lo que has demostrado: la cifra del MJD-102 era correcta, pero **ahora lo sabes**. Antes era una cifra con una advertencia; ahora es una cifra comprobada. Eso se le dice a Marcos.

Para que Marcos lo pueda ejecutar él, como pedía, el siguiente paso es pasarlo a un script con la ruta del fichero como argumento (10.2), y que no necesite abrir un notebook. Y una cosa más: las 9 referencias `OBS-` cuentan aquí como referencias de Guarnizo. Son reales (están en el almacén), pero no tienen ficha; dilo en una línea.

MJD-502 · Probar la normalización del SKU

Dos partes: probar la función con casos pensados a propósito, y compararla con silver sobre los datos reales.

Los casos raros. Un `assert` comprueba que algo es verdad y, si no lo es, para con un error. Es la forma más sencilla de escribir pruebas:

PYTHON · EN TU NOTEBOOK

```
casos = {
    "CER-0001": "CER-0001",     # ya correcto: no debe cambiar
    "cer-0012": "CER-0012",     # minúsculas
    " CER-0012": "CER-0012",    # espacio delante
    "CER0007 ": "CER-0007",     # sin guion y espacio detrás
    "obs-0003": "OBS-0003",     # referencia antigua: también se normaliza
}

for entrada, esperado in casos.items():
    obtenido = mu.normalizar_sku(entrada)
    assert obtenido == esperado, f"{entrada!r}: esperaba {esperado!r} y salió {obtenido!r}"

print(f"{len(casos)} casos correctos")
```

RESULTADO

```
5 casos correctos
```

Y un caso que la regla **no** sabe resolver, para saber dónde están sus límites:

PYTHON · EN TU NOTEBOOK

```
assert mu.normalizar_sku("CER-12") == "CER-0012", "un SKU con ceros de menos no se arregla"
```

ERROR

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[3], line 1
----> 1 assert mu.normalizar_sku("CER-12") == "CER-0012", "un SKU con ceros de menos no se arregla"

AssertionError: un SKU con ceros de menos no se arregla
```

La función devuelve `CER-12`, no `CER-0012`. Ningún SKU del inventario está escrito así, así que hoy no importa; pero ahora está documentado qué hace la regla y qué no.

La comparación con silver. Para cada fila del inventario, el SKU normalizado por Python frente al de `silver.inventario`, que se construyó con la regla de SQL. Las dos listas están en el mismo orden si se ordenan igual; más seguro es comparar por la clave completa de la fila (almacén, SKU original y ubicación):

PYTHON · EN TU NOTEBOOK

```
import duckdb

con = duckdb.connect("../db/marejada.duckdb", read_only=True)
silver = con.sql("""
    SELECT b.almacen, b.sku AS sku_original, b.ubicacion, s.sku AS sku_silver
    FROM bronze.inventario AS b
    JOIN silver.inventario AS s
      ON s.almacen = b.almacen AND s.ubicacion = b.ubicacion AND s.descripcion = b.descripcion
""").fetchall()
con.close()

distintos = [(a, orig, sil) for a, orig, ubi, sil in silver if mu.normalizar_sku(orig) != sil]
print(f"Filas comparadas: {len(silver)}")
print(f"Filas en las que Python y SQL no coinciden: {len(distintos)}")
```

RESULTADO

```
Filas comparadas: 381
Filas en las que Python y SQL no coinciden: 0
```

381 filas, cero diferencias: las dos implementaciones hacen lo mismo con todos los datos reales.

REVISIÓN DE ELENA

Esto es lo que se llama **pruebas**, y acabas de hacer los dos tipos que importan: pruebas de casos pensados a mano (que documentan qué hace la función y qué no) y una comparación con los datos reales contra la otra implementación. Lo más valioso es el `assert` que falla: ahora cualquiera sabe que `CER-12` no se corrige, sin tener que descubrirlo en producción.

Un paso más, para cuando tengas tiempo: estos `assert` pueden vivir en un fichero `tests/test_marejada_utils.py` y ejecutarse todos con una herramienta llamada `pytest`. Es lo que verás en el Bloque 7 (MLOps). La idea ya la tienes.

MJD-503 · Mensajes de reactivación

La lista sale de gold (SQL hace bien lo que hace bien) y el texto se construye en Python (f-strings y ficheros). El límite de días va como parámetros (9.3), no metido en el texto de la consulta:

PYTHON · EN TU NOTEBOOK

```
con = duckdb.connect("../db/marejada.duckdb", read_only=True)
clientes = con.execute("""
    SELECT c.nombre_comercial,
           c.comercial_asignado,
           DATE '2026-09-30' - max(d.fecha) AS dias_sin_pedir
    FROM gold.fact_ventas AS f
    JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
    JOIN gold.dim_fecha   AS d ON d.fecha_key   = f.fecha_key
    WHERE c.provincia = 'Cantabria' AND c.es_version_actual
    GROUP BY c.cliente_id, c.nombre_comercial, c.comercial_asignado
    HAVING dias_sin_pedir BETWEEN ? AND ?
    ORDER BY dias_sin_pedir
""", [90, 180]).fetchall()
con.close()

print(f"{len(clientes)} clientes")
print(clientes[0])
```

RESULTADO

```
18 clientes
('Supermercado El Botxo', 'Lorena Arce Díaz', 91)
```

18 clientes de Cantabria entre 90 y 180 días sin pedir. Fíjate en `c.es_version_actual`: el comercial que firma el mensaje es el de **hoy**, no el que llevaba al cliente hace un año (la SCD del Volumen 04). Ahora, los mensajes:

PYTHON · EN TU NOTEBOOK

```
def mensaje(cliente, comercial, dias):
    nombre_comercial = comercial.split()[0]
    semanas = dias // 7
    return (
        f"Hola, equipo de {cliente}. Soy {nombre_comercial}, de Marejada. "
        f"Hace {semanas} semanas que no os vemos por aquí y queríamos saber si todo va bien. "
        f"Si necesitáis algo para esta temporada, escribidme por aquí y os lo preparo."
    )


textos = [mensaje(nombre, comercial, dias) for nombre, comercial, dias in clientes]

with open("../entregables/mjd-503_mensajes.txt", "w", encoding="utf-8") as f:
    f.write("\n\n".join(textos) + "\n")

print(textos[0])
```

RESULTADO

```
Hola, equipo de Supermercado El Botxo. Soy Lorena, de Marejada. Hace 13 semanas que no os vemos por aquí
y queríamos saber si todo va bien. Si necesitáis algo para esta temporada, escribidme por aquí y os lo
preparo.
```

DESMONTANDO EL COMANDO

`comercial.split()[0] · dias // 7 · "\n\n".join(textos)`

`comercial.split()[0]`	El nombre de pila: `split()` sin argumentos parte por los espacios, y `[0]` coge el primer trozo. "Celia Gómez Sáiz" → "Celia".
`dias // 7`	Las semanas completas, con la división entera del 2.2. "Hace 13 semanas" suena más natural que "hace 95 días".
`( f"..." f"..." )`	Varios textos seguidos dentro de paréntesis se juntan en uno. Así un mensaje largo no se sale de la pantalla.
`"\n\n".join(textos)`	Une todos los mensajes en un solo texto, con una línea en blanco entre cada dos. `join` es el `string_agg` del Volumen 02.

REVISIÓN DE ELENA

Bien hecho, y bien repartido entre SQL y Python. Antes de dárselo a Raquel, dos cosas que miraría:

- **Lee cinco mensajes en voz alta.** Un mensaje automático que suena automático no funciona. "Hola, equipo de Supermercado El Botxo" queda raro; quizá Raquel prefiera que cada comercial lo adapte. Pregúntale.
- **Los datos personales.** Estás generando un fichero con nombres de clientes y comerciales. En una empresa real, eso no va a GitHub. Añade `entregables/*.txt` al `.gitignore`, o mejor, guarda estos ficheros fuera del repositorio. Aquí los datos son inventados, pero la costumbre es la de siempre (Volumen 01, 7.4).

MJD-504 · El organigrama como árbol

Primero, de cada jefe, la lista de las personas que dependen de él. Después, una función que dibuja a una persona y **se llama a sí misma** para cada una de las que dependen de ella. Una función que se llama a sí misma es **recursiva**: es la CTE recursiva del Volumen 03 (3.5), en Python.

PYTHON · EN TU NOTEBOOK

```
with open("../datos/rrhh/comerciales.csv", encoding="latin-1", newline="") as f:
    personas = list(csv.DictReader(f))

nombre = {p["comercial_id"]: f"{p['nombre']} {p['apellidos']}" for p in personas}
puesto = {p["comercial_id"]: p["puesto"] for p in personas}
de_baja = {p["comercial_id"] for p in personas if p["fecha_baja"]}

subordinados = {}
for p in personas:
    subordinados.setdefault(p["jefe_id"], []).append(p["comercial_id"])


def dibujar(persona_id, nivel=0):
    marca = "  (de baja)" if persona_id in de_baja else ""
    print("    " * nivel + f"{nombre[persona_id]} · {puesto[persona_id]}{marca}")
    for sub in subordinados.get(persona_id, []):
        dibujar(sub, nivel + 1)


for raiz in subordinados[""]:
    dibujar(raiz)
```

RESULTADO

```
Íñigo Sainz Ruiz · Director comercial
    Raquel Fernández Cobo · Jefa de ventas Cantabria
        Daniel Gutiérrez Pérez · Comercial
        Sara Ceballos Ortiz · Comercial
        Pablo Revuelta Gómez · Comercial
        Lorena Arce Díaz · Comercial
        Héctor Bustamante Lavín · Comercial  (de baja)
        Celia Gómez Sáiz · Comercial
    Nuria Álvarez Menéndez · Jefa de ventas Norte
        Adrián Suárez Fernández · Comercial
        Ane Etxeberria Zubiri · Comercial
        Mikel Goikoetxea Arana · Comercial
    Rubén Martín Calvo · Comercial
    Óscar Peña Revilla · Comercial  (de baja)
```

DESMONTANDO EL COMANDO

`subordinados.setdefault(p["jefe_id"], []).append(...)` · `subordinados[""]` · `dibujar(sub, nivel + 1)`

`subordinados`	Un diccionario: para cada `jefe_id`, la lista de sus subordinados. El mismo `setdefault` del MJD-501, con listas.
`subordinados[""]`	Quien tiene el `jefe_id` vacío (en el CSV, un texto vacío, no `None`): la raíz del árbol. Íñigo.
`dibujar(sub, nivel + 1)`	La llamada recursiva: dibuja a cada subordinado un nivel más adentro. Para cuando una persona no tiene subordinados (`.get(persona_id, [])` devuelve una lista vacía y el `for` no hace nada).
`"    " * nivel`	Un texto multiplicado por un número se repite: cuatro espacios por cada nivel.

REVISIÓN DE ELENA

Correcto, y la recursión bien parada. El árbol coincide con el de la CTE recursiva del Volumen 03, que es la comprobación por otra vía.

Para el manual, quita a los que están de baja: el organigrama de bienvenida debe mostrar la empresa de hoy. Es una línea: `if persona_id in de_baja: return` al principio de `dibujar`. Lo has marcado en lugar de quitarlo, que para ti como analista es mejor (se ve todo), pero el documento es para otra persona.

MJD-505 · Los pedidos web que no llegaron al ERP

El JSON con el módulo `json` (7.6), y los pedidos con estado `sync_error`:

PYTHON · EN TU NOTEBOOK

```
import json
from collections import Counter

with open("../datos/web/pedidos_web.json", encoding="utf-8") as f:
    web = json.load(f)

perdidos = [p for p in web["orders"] if p["status"] == "sync_error"]


def importe(pedido):
    lineas = sum(i["qty"] * i["unit_price"] * (1 - i["discount"] / 100) for i in pedido["items"])
    return lineas + pedido["shipping"]


total = sum(importe(p) for p in perdidos)
clientes = {p["customer"]["nif"] for p in perdidos}
por_mes = Counter(p["created_at"][:7] for p in perdidos)

print(f"Pedidos que no llegaron al ERP: {len(perdidos)}")
print(f"Importe: {total:,.2f} € (con portes)")
print(f"Clientes distintos: {len(clientes)}")
print(f"Primero: {min(p['created_at'] for p in perdidos)}")
print(f"Último:  {max(p['created_at'] for p in perdidos)}")
print("Por mes:", dict(sorted(por_mes.items())))
```

RESULTADO

```
Pedidos que no llegaron al ERP: 31
Importe: 11,517.21 € (con portes)
Clientes distintos: 31
Primero: 2025-05-13T21:40:00Z
Último:  2026-08-31T23:24:00Z
Por mes: {'2025-05': 1, '2025-06': 1, '2025-07': 2, '2025-08': 1, '2025-09': 2, '2025-10': 4, '2025-11':
2, '2025-12': 2, '2026-01': 2, '2026-02': 3, '2026-03': 2, '2026-04': 2, '2026-05': 3, '2026-06': 2,
'2026-08': 2}
```

DESMONTANDO EL COMANDO

`p["created_at"][:7]`

`p["created_at"]`	La fecha y hora del pedido como texto: `"2025-04-07T04:04:00Z"`.
`[:7]`	Los siete primeros caracteres: `"2025-04"`, el año y el mes. Con fechas en formato ISO (año-mes-día), recortar el texto es la forma más rápida de agrupar por mes. Con fechas en formato español (día/mes/año) no funcionaría.
`Counter( ... )`	Cuenta cuántos pedidos hay de cada mes.
`dict(sorted(por_mes.items()))`	Ordena por mes para imprimirlos en orden.

La respuesta a Paula: 31 pedidos de 31 clientes distintos, por 11.517,21 € con portes, entre mayo de 2025 y agosto de 2026. Casi todos los meses hay uno o varios (solo julio de 2026 se libra): no es un fallo puntual, es un goteo constante. Cada uno es un cliente que hizo un pedido que nunca se sirvió ni se facturó.

REVISIÓN DE ELENA

Bien. Este es de los tickets que se convierten en un problema de empresa, así que antes de mandarlo, piensa en lo que va a pasar después:

- **Son clientes que pidieron y no recibieron nada.** Cada uno es una queja posible, o un cliente que se fue sin quejarse. Paula tiene que saber quiénes son: añade la lista de NIF y nombres.
- **Un goteo de uno o dos al mes durante año y medio** quiere decir que nadie mira el estado `sync_error`. La solución no es tu informe, es una alerta: un script como el del capítulo 10 que se ejecute cada día y avise si aparece uno nuevo. Proponlo.
- Ojo con el importe: está calculado con los precios del JSON, que es lo que vio el cliente. No es lo que facturaría el ERP (que aplicaría el descuento habitual del cliente). Dilo.

MJD-506 · El informe semanal

Un script con un argumento (la fecha del lunes), que calcula las dos semanas con `datetime` y pasa las fechas como parámetros a la consulta:

FICHERO · scripts/informe_semana.py

```
"""Ventas de una semana por canal, comparadas con la misma semana del año anterior.

Uso:  uv run python scripts/informe_semana.py 2026-09-21
"""
import argparse
import datetime as dt

import duckdb

CONSULTA = """
    SELECT f.canal, round(sum(f.importe_neto), 2) AS ventas
    FROM gold.fact_ventas AS f
    JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
    WHERE d.fecha BETWEEN ? AND ?
    GROUP BY f.canal
"""


def ventas_por_canal(con, lunes):
    domingo = lunes + dt.timedelta(days=6)
    return dict(con.execute(CONSULTA, [lunes, domingo]).fetchall())


def main():
    parser = argparse.ArgumentParser(description="Informe semanal de ventas por canal.")
    parser.add_argument("lunes", help="Lunes de la semana, en formato AAAA-MM-DD")
    args = parser.parse_args()

    lunes = dt.date.fromisoformat(args.lunes)
    if lunes.isoweekday() != 1:
        parser.error(f"{lunes} no es lunes")
    # el lunes de la misma semana ISO del año anterior
    anio, semana, _ = lunes.isocalendar()
    lunes_anterior = dt.date.fromisocalendar(anio - 1, semana, 1)

    con = duckdb.connect("db/marejada.duckdb", read_only=True)
    actual = ventas_por_canal(con, lunes)
    anterior = ventas_por_canal(con, lunes_anterior)
    con.close()

    print(f"Semana {semana} de {anio} (del {lunes:%d/%m} al {lunes + dt.timedelta(days=6):%d/%m})")
    print(f"comparada con la semana {semana} de {anio - 1} (desde el {lunes_anterior:%d/%m/%Y})\n")
    print(f"{'Canal':<12}{'Ventas':>14}{'Año anterior':>16}{'Variación':>12}")
    for canal in sorted(set(actual) | set(anterior)):
        v, a = actual.get(canal, 0), anterior.get(canal, 0)
        variacion = f"{100 * (v / a - 1):+.1f} %" if a else "nuevo"
        print(f"{canal:<12}{v:>14,.2f}{a:>16,.2f}{variacion:>12}")
    v, a = sum(actual.values()), sum(anterior.values())
    print(f"{'Total':<12}{v:>14,.2f}{a:>16,.2f}{100 * (v / a - 1):>+10.1f} %")


if __name__ == "__main__":
    main()
```

TÚ ESCRIBES

```
uv run python scripts/informe_semana.py 2026-09-21
uv run python scripts/informe_semana.py 2026-09-23
```

RESULTADO

```
Semana 39 de 2026 (del 21/09 al 27/09)
comparada con la semana 39 de 2025 (desde el 22/09/2025)

Canal              Ventas    Año anterior   Variación
Comercial       178,291.34      226,137.14     -21.2 %
Teléfono         76,351.81       72,075.25      +5.9 %
Web B2B          91,153.40       45,015.05    +102.5 %
Total           345,796.55      343,227.44      +0.7 %
usage: informe_semana.py [-h] lunes
informe_semana.py: error: 2026-09-23 no es lunes
```

La semana 39 de 2026 vende casi lo mismo que la de 2025 (+0,7 %), pero con una mezcla muy distinta: la web dobla sus ventas y la visita comercial cae un 21 %. Es la historia del MJD-301 del Volumen 03, vista en una sola semana. Y el segundo caso: el miércoles 23 no es lunes, y el script lo dice en lugar de inventarse una semana.

DESMONTANDO EL COMANDO

`dt.date.fromisocalendar(anio - 1, semana, 1)`

`lunes.isocalendar()`	Devuelve el año, la semana y el día de la semana según la norma ISO (la de `semana_iso` en `dim_fecha`). El `_` recoge el día, que no hace falta: es la convención para "un valor que no voy a usar".
`fromisocalendar(anio - 1, semana, 1)`	El lunes (día 1) de la misma semana del año anterior. Comparar **semana con semana** y no fecha con fecha (el 21 de septiembre de 2025 fue domingo) es lo que hace justa la comparación: lunes con lunes.
`parser.error(...)`	Si la fecha no es un lunes, el script para con un mensaje claro y el uso correcto. Mejor que un informe de una semana a medias.
`dict(con.execute(...).fetchall())`	Una lista de tuplas de dos elementos se convierte directamente en un diccionario: canal → ventas.
`for canal in sorted(...)`	Recorre `set(actual) | set(anterior)`: la unión de los dos conjuntos de canales (3.5), todos los que aparecen en cualquiera de las dos semanas. Así un canal que no vendió en una de ellas sale igual, con 0.

REVISIÓN DE ELENA

Muy bien: la validación de la fecha, la comparación por semana ISO y el canal que podría no existir el año anterior. Esto ya se puede programar para que se ejecute solo cada lunes.

Dos detalles para la siguiente versión. El primero, el formato de los números: `2,142,228.00` es el estilo inglés, y a Íñigo le saldrá raro; en el Volumen 06 verás cómo dar formato español. El segundo, más importante: el script asume que la base de datos está al día. El día que se ejecute un lunes y la carga del fin de semana haya fallado, mostrará una semana incompleta sin avisar. Un informe automático debería comprobar primero que hay datos del último día de la semana.

APÉNDICES

Para tener a mano

La chuleta del lenguaje y lo que viene en el Volumen 06.

A. Chuleta de Python

B. Qué viene en el Volumen 06

A. Chuleta de Python

Valores y texto

NECESITAS	ESCRIBE
Tipo de un valor	`type(x)`
Convertir	`int("42")` · `float("41.65")` · `str(42)` · `Decimal("41.65")`
División normal / entera / resto / potencia	`7 / 2` · `7 // 2` · `7 % 2` · `2 ** 10`
Limpiar texto	`t.strip()` · `t.upper()` · `t.lower()` · `t.title()` · `t.replace("-", "")`
Partir y unir	`t.split("-")` · `"-".join(lista)`
Trozo de texto o lista	`t[:3]` · `t[3:]` · `lista[-1]`
Texto con valores	`f"{x:,.2f} €"` · `f"{nombre:<12}"` · `f"{n:>8}"`
¿Es nulo?	`x is None` · `x is not None`

Colecciones

NECESITAS	ESCRIBE
Lista: añadir, quitar, buscar	`l.append(x)` · `l.remove(x)` · `x in l`
Copiar una lista (de verdad)	`l.copy()`
Diccionario: leer sin error	`d.get("clave", valor_por_defecto)`
Diccionario: acumular	`d[k] = d.get(k, 0) + 1` · `d.setdefault(k, []).append(x)`
Recorrer un diccionario	`for clave, valor in d.items():`
Valores distintos	`set(lista)` · `a - b` · `a & b` · `a | b`
Contar	`Counter(lista)` · `.most_common(3)`
Ordenar por un criterio	`sorted(l, key=lambda x: x[1], reverse=True)`
Lista nueva a partir de otra	`[f(x) for x in l if condición]`

Control de flujo y funciones

NECESITAS	ESCRIBE
Decidir	`if ...:` · `elif ...:` · `else:`
Recorrer	`for x in l:` · `range(n)` · `enumerate(l, start=1)` · `zip(a, b)`
Función	`def f(a, b=0) -> float:` y `return`
Parámetro opcional con lista	`def f(x, l=None):` y `if l is None: l = []`

NECESITAS	ESCRIBE
Atrapar un error concreto	`try: ... except ValueError: ...`
Lanzar un error	`raise ValueError("mensaje")`
Probar	`assert condición, "mensaje si falla"`

Ficheros, módulos y DuckDB

NECESITAS	ESCRIBE
Ruta	`Path("../datos") / "erp" / "pedidos.csv"`
Leer texto	`with open(ruta, encoding="utf-8") as f: texto = f.read()`
Escribir	`open(ruta, "w", encoding="utf-8")` · `"a"` para añadir
CSV	`csv.DictReader(f, delimiter=";")` con `open(..., newline="")`
JSON	`json.load(f)` · `json.dump(datos, f, ensure_ascii=False)`
Fechas	`dt.date(2026, 9, 30)` · `dt.timedelta(days=90)` · `strptime` · `strftime`
Importar	`import json` · `from pathlib import Path` · `import datetime as dt`
Instalar un paquete	`uv add requests` (en la terminal)
Conectar	`con = duckdb.connect("../db/marejada.duckdb", read_only=True)`
Filas como tuplas	`con.sql("...").fetchall()` · `.fetchone()`
Con parámetros	`con.execute("... WHERE x = ?", [valor]).fetchall()`
Script	`def main(): ...` y `if __name__ == "__main__": main()`
Argumentos	`argparse.ArgumentParser()` · `add_argument("x")` · `add_argument("--y", type=int)`

Las trampas de este volumen

TRAMPA	CÓMO SE EVITA
`0.1 + 0.2` no es `0.3`	`Decimal` para dinero, o redondear al final
`texto.strip()` no cambia `texto`	Guardar el resultado: `texto = texto.strip()`
`copia = lista` no copia	`lista.copy()`
`def f(x, l=[])`	`l=None` y crear la lista dentro
`if descuento:` con descuento 0	`if descuento is not None:`
`except:` a secas	Atrapar el tipo concreto: `except ValueError:`
Abrir sin `encoding`	Escribirlo siempre
Cambiar un módulo y no ver el cambio	Reiniciar el kernel
Valores en una consulta con f-strings	Parámetros con `?`

B. Qué viene en el Volumen 06

El Volumen 06 cierra el Bloque 0 con las herramientas de datos de Python:

- **pandas**: la tabla de Python. Leer los CSV, el Excel y el JSON de Marejada, filtrar, agrupar, unir, y por qué a veces es mejor DuckDB y a veces pandas.
- **Ficheros de datos modernos**: Parquet, el formato por columnas que usan Fabric, Databricks y AWS.
- **APIs**: pedir datos a un servicio web con `requests`, paginación, errores y límites.
- **Gráficos** con matplotlib, lo justo para explorar.
- El cruce que quedó pendiente en el MJD-208: los pedidos de la tienda web frente a los del ERP.

ANTES DE PASAR AL VOLUMEN 06

Tienes que poder hacer esto sin mirar: leer un CSV con el módulo `csv` y acumular por un campo con un diccionario, escribir una función con un parámetro opcional sin caer en la trampa de la lista, atrapar un `ValueError` concreto, lanzar una consulta a gold con parámetros y recorrer sus filas, y un script con `main()` y un argumento. Y los seis tickets en `notebooks/` y `scripts/`, en GitHub.